

System Design Interview

URL Shortening Service | Mock Google-style HLD + LLD Session

1. Requirements Gathering

Functional Requirements (final)

- Shorten a long URL into a unique short code, with optional custom alias
- Redirect a short URL to its original long URL (HTTP 302)
- Support configurable expiration (TTL) per link
- Allow verifying/looking up the long URL for a short code without redirecting
- (Optional) basic click analytics

Non-Functional Requirements (final)

Attribute	Target	Rationale
Read : Write ratio	100 : 1	Redirects vastly outnumber URL creations
Availability	99.99%	~52 minutes downtime/year; needs multi-AZ redundancy
Latency	Low (sub-100ms goal)	Redirect UX must feel instant
Consistency	Eventual	New links need not be visible on every node instantly

Interviewer note: the initial draft listed vague NFRs like “scalable” and “99% uptime” without supporting numbers. Real requirements need concrete, defensible figures — these drive every later architecture decision.

2. Capacity Estimation

Traffic (QPS)

Metric	Average	Peak (2x)
Write QPS	116 / sec	232 / sec
Read QPS	11,600 / sec	23,200 / sec

Based on an assumed 10M new short URLs created per day, with the agreed 100:1 read:write ratio. The high peak read QPS (23,200/sec) is the number that justifies adding a caching layer in front of the database.

Storage

Assumption	Value
New URLs / day	10,000,000
Retention period	5 years
Total records (5 yrs)	18.25 billion
Avg. record size	500 bytes
Total storage	~9.125 TB

Key takeaway: 9.1 TB is not large by modern standards, but 18+ billion rows is large enough that sharding decisions should be driven by index/query performance at that row count, not by raw disk size.

3. Short Code Generation Strategy

Code length

Using Base62 encoding (a-z, A-Z, 0-9 = 62 characters), $62^7 \approx 3.5$ trillion possible codes — comfortably above the 18.25 billion records projected over 5 years, with wide headroom for growth. **7 characters** was selected as the code length.

Generation approach: range-based distributed ID allocation

Two approaches were discussed for generating globally unique IDs across multiple stateless app servers:

Approach	How it works	Verdict
Hash-based (MD5/SHA + truncate)	Hash the long URL, take first 7 chars; collision risk	Rejected. Requires DB lookup and needs a lookup before encode
Range-based counter (chosen)	A central counter hands out blocks of IDs (e.g. 1000 at a time) to each app server	Selected. A write volume of 232/sec is lower enough to fit in memory
Snowflake-style ID	timestamp + machine ID + local sequence, generated independently	Considered but rejected due to added complexity (machine ID)

Cache eviction policy

LRU (Least Recently Used) was selected for the Redis cache layer. It naturally adapts to trending/viral links and is simple to implement (Redis's built-in *allkeys-lru* policy). LFU (Least Frequently Used) is theoretically a tighter fit for the expected Zipfian access pattern (small % of links get most clicks), but LRU was preferred for its simplicity and faster reaction to sudden popularity spikes.

4. High-Level Design (HLD)

The finalized architecture, in request-flow order:

Component	Role
Client	Browser or API consumer
Load balancer	Distributes traffic across stateless app servers; needed for the 99.99% availability target
App servers	Stateless, horizontally scaled; handle both create and redirect requests
ID range allocator	Hands out blocks of unique numeric IDs for short-code generation (write path)
Redis cache	Cache-aside lookup of short code → long URL (read path); LRU eviction
Database (primary + read replicas)	Source of truth; primary handles writes, replicas absorb read load on cache misses

Write path

Client → Load balancer → App server → requests a new ID from the ID range allocator → writes {short_code, long_url} to the primary DB → primes the Redis cache → returns the short URL to the client.

Read path

Client → Load balancer → App server → checks Redis cache first. **Cache hit:** respond immediately with a 302 redirect, no DB touched. **Cache miss:** query a DB read replica, populate the cache, then respond with the 302 redirect.

Interviewer note: an earlier draft of this diagram described generic internet routing (DNS resolution) instead of the system's own internal components, and omitted the cache layer entirely despite the 23,200 peak read QPS explicitly calling for one. The revised version above was rated 5/5.

5. Low-Level Design (LLD)

Database schema – short_url table

Column	Type	Notes
id	BIGINT (PK)	From the range allocator, not DB auto-increment
short_code	VARCHAR(10)	Unique index – required for fast redirect lookups
long_url	VARCHAR(2048)	The original URL
custom_alias	VARCHAR(50)	Nullable; user-chosen alternative to the generated code
status	VARCHAR(20)	ACTIVE / EXPIRED / DELETED / BLOCKED – replaces a vague boolean
expiry_date	TIMESTAMP	Nullable = never expires
created_by	BIGINT	Nullable if anonymous creation is allowed
created_at / updated_at	TIMESTAMP	Audit timestamps

A separate short_url_audit table tracks CREATED / UPDATED / DELETED actions for traceability.

API contracts

Endpoint	Purpose	Success status	Failure statuses
POST /api/v1/urls	Create a short URL	201 Created	409 Conflict (alias taken), 400 Bad Request
GET /{shortCode}	Redirect to the long URL	302 Found	404 Not Found, 410 Gone (expired/deleted)
GET /api/v1/urls/{shortCode}	Verify/inspect mapping without redirecting	200 OK	404 Not Found, 410 Gone

Key design decisions and their reasoning

302 over 301 for redirects

301 responses get cached by the browser, so subsequent clicks never reach the server again — breaking click analytics and preventing expiry/updates from taking effect. 302 keeps every click server-visible.

404 vs 410

404 means the short code never existed. 410 Gone means it existed but is now permanently unavailable (expired/deleted/blocked). Distinguishing them helps debugging, informs crawler behavior, and produces more useful analytics on dead-link traffic.

Custom alias collisions

Handled via a database-level unique constraint, not a ‘check availability, then insert’ pattern — the latter has a race condition (TOCTOU) where two concurrent requests can both pass the check before either commits, producing duplicate aliases.

Rate limiting placement

Enforced at the load balancer / API gateway layer using a token-bucket algorithm, so abusive traffic is rejected before it consumes app server or database resources. Write (create) endpoints are throttled far more aggressively than read (redirect) endpoints, since a viral link can legitimately generate thousands of redirects without thousands of new link creations.

Click analytics without slowing redirects

Click events are published fire-and-forget to a message queue and processed asynchronously by a separate consumer, so the redirect response is never blocked waiting on analytics writes. This also means an analytics outage cannot take down redirects.

6. Overall Interview Performance Summary

Stage	Score	Notes
Functional / NFR gathering	2.5 → 4.5 / 5	Started vague, refined well once pushed for concrete numbers
Capacity estimation	4 / 5 (avg)	One arithmetic slip (self-corrected), otherwise clean
Algorithm design (short code)	3.5 / 5 (avg)	Diagnosed problems well; needed prompting for solutions
High-level design	2.5 → 5 / 5	Initial diagram missed cache/load balancer; final version solid
Low-level design (schema, API)	3 / 5 (avg)	Reasonable structure; missed TTL field and status enum initially
Edge cases (redirects, errors, collisions, rate limiting, analytics)	4 / 5 (avg)	Correctly reasoned through concurrency and independently picked

What went well

- Good self-correction instinct — caught an arithmetic mistake in the storage calculation without prompting
- Correctly reasoned through the concurrency / race-condition trap in custom alias handling
- Independently proposed the correct async queue pattern for click analytics, without being given multiple-choice options

Areas to improve

- Drive more of the design without prompts — several strong answers only emerged once given multiple-choice options; practice generating and justifying options independently
- Carry earlier decisions forward consistently — the TTL/expiry requirement was agreed early but dropped from the first draft of the database schema
- Pair each design choice with an explicit 'why' — the reasoning behind a decision is often weighted more heavily than the decision itself in a real interview

Note: this summary excludes the Spring Boot implementation code produced in the same session. A separate, runnable project (entities, repositories, services, controllers, Redis cache config, rate limiter, and DB schema) implementing every decision above was delivered as a downloadable zip.